HDOC DAY–70

Recursive Function to Calculate the Digital Root

Aim

To write a C program that calculates the digital root of a number using recursion and uses a static variable to count the total number of function calls.

Concept

The **digital root** is obtained by repeatedly adding the digits of a number until a single-digit number remains. A recursive function calls itself with the sum of the digits. A **static variable** retains its value between function calls, so it can keep a running count.

Algorithm

1. Start.
2. Read a positive integer n.
3. Call the function digitalRoot(n).
4. In the function, increment the static call counter.
5. If n is a single-digit number, display the call count and return n.
6. Otherwise, calculate the sum of its digits.
7. Call digitalRoot() again with this sum.
8. Display the returned digital root and stop.

C Program

```
#include <stdio.h>

int digitalRoot(int n)
{
    static int calls = 0;
    int sum = 0;

    calls++;

    if (n < 10)
    {
        printf("Total function calls = %d\n", calls);
        return n;
    }

    while (n > 0)
    {
        sum = sum + (n % 10);
        n = n / 10;
    }

    return digitalRoot(sum);
}

int main()
{
    int n, result;

    printf("Enter a positive number: ");
```

```
    scanf("%d", &n);

    if (n < 0)
    {
        printf("Please enter a non-negative number.\n");
        return 0;
    }

    result = digitalRoot(n);

    printf("Digital root = %d\n", result);

    return 0;
}
```

Sample Output

```
Enter a positive number: 9875
Total function calls = 3
Digital root = 2
```

```
Enter a positive number: 9875
Total function calls = 4
Digital root = 2
```

9875 → 9 + 8 + 7 + 5 = 29
29 → 2 + 9 = 11
11 → 1 + 1 = 2

The function is called with 9875, 29, 11 and 2. Therefore, the static counter displays **4** total function calls.

Result

The C program successfully calculates the digital root using recursion. The static variable counts and retains the total number of function calls.

Note: The counter includes the first call from main() as well as the recursive calls.